

Phase 12: Cookies & Sessions Documentation

1. Cookies

Cookies bhaneko client-side (browser) ma store hune sanno text data ho. Yesle server lai limited memory dinchha jasle user request-haru track garna sakchha. [[1](#), [2](#)]

Core Concepts:

- **Stateless HTTP:** HTTP protocol-le purano requests birsinchha, cookies-le data yaad rakhna help garchha.
 - **Storage:** Browser ma key-value pair ko roop ma baschha.
 - **Automatic Send:** Harek choti browser-le server lai request pathauda, tyo domain ko cookie aafai sangai janchha. [[1](#), [2](#), [3](#)]
-

2. cookie-parser

Express.js ma aayeko cookies lai direct read garna mildaina. `cookie-parser` middleware le request ko headers bata cookie block lai easily readable object (`req.cookies`) ma convert garchha.

Implementation:

javascript

```
const express = require('express');
const cookieParser = require('cookie-parser');

const app = express();
app.use(cookieParser()); // Middleware initialized

// Setting a cookie
app.get('/set-cookie', (req, res) => {
  res.cookie('username', 'Ram', { maxAge: 900000 }); // 15 mins
  res.send('Cookie has been saved!');
});

// Reading a cookie
app.get('/get-cookie', (req, res) => {
  const user = req.cookies.username;
  res.send(`Welcome back, ${user}`);
});
```

Use code with caution.

3. express-session

Cookies browser ma basne bhayeko ^{ले} yesma sensitive data (jastai user id, password) rakhnu unsafe hunchha. Session-le data **Server-side** ma store garchha ra client lai euta unique `Session ID` (euta safe string) matrai cookie ko roop ma dinchha. [[1](#), [2](#), [3](#), [4](#), [5](#)]

Implementation:

javascript

```
const session = require('express-session');

app.use(session({
  secret: 'mySuperSecretKey', // Session encrypt garna use hunchha
  resave: false,              // Data change nabhaye pani feri save
                                // nagarne
  saveUninitialized: true,     // Empty session pani save garne
  cookie: { secure: false }    // HTTP ko lagi false, HTTPS ko lagi true
}));

// Accessing session data
app.get('/login', (req, res) => {
  req.session.user = { id: 101, role: 'admin' };
  res.send('Logged in and session created!');
});
```

Use code with caution.

4. Session Storage

By default, `express-session` le session data server ko temporary memory (**MemoryStore**) ma save garchha. Server restart huda sabai user log out hunchhan. Production ma yeslai roknai permanent database (Session Storage) chainchha. [[1](#)]

Popular Storages:

- **Redis:** Sabai bhanda fast (In-memory database).
 - **MongoDB:** `connect-mongo` package use garera session database mai permanent store garna sakinchha.
-

5. Secure Cookies

Yo euta cookie setting/attribute ho jasले cookie lai secure connection (HTTPS) bata matrai server ma pathauna allow garchha. [\[1\]](#), [\[2\]](#), [\[3\]](#)

Security Benefit:

- **MitM Attack Prevention:** कसैले network traffic header lookup garyo bhane pani HTTP simple plain text network ma cookie leak hudaina. [\[1\]](#)

javascript

```
res.cookie('token', 'xyz123', {  
  secure: true // Server strictly HTTPS chaleko huna parchha  
});
```

Use code with caution.

6. HttpOnly Cookies

Yo sabaibhanda mahatwapurna security setting ho. `httpOnly: true` gareko cookie lai client-side JavaScript (`document.cookie`) le read garna mildaina.

Security Benefit:

- **XSS Attack Prevention:** Cross-Site Scripting (XSS) attack bhayera hacker le site ma malicious script run garyo bhane pani user ko main session token chori garna sakdaina. [\[1\]](#), [\[2\]](#), [\[3\]](#), [\[4\]](#)

javascript

```
res.cookie('session_id', 'secret_token_here', {  
  httpOnly: true, // Browser JS le read garna mildaina  
  secure: true    // HTTPS connection matrai chahine  
});
```